qualifications, experience, and so on. So the CEO asks the hiring manager for help in processing the resumes.

The hiring manager divides the resumes into smaller lots containing only a hundred resumes per lot, and requests for a few assistants from the HR department. Once the HR department allocates the assistants, the hiring manager distributes the lots to them for filtering. Each assistant processes only the resumes in the assigned lots, and submits the shortlisted resumes to the hiring manager.

Once all the assistants complete the processing, the hiring manager collates all the shortlisted resumes and gives the bundle to the CEO.

We can deduce the following observations from this analogy:

1. The work is to shortlist a few resumes from the thousands of resumes.
2. The hiring manager divides the work into smaller tasks.
3. The hiring manager requests for assistants from the HR department.
4. The hiring manager distributes the tasks to the assistants.
5. Each task consists of processing only a smaller set of resumes.
6. The assistants quickly finish the assigned tasks since they have only a few resumes to process.
7. The assistants submit the shortlisted resumes to the hiring manager.
8. The hiring manager collates all the shortlisted resumes from all the assistants.
9. The hiring manager submits the shortlisted resumes to the CEO.

In the Spark architecture, as illustrated in Figure 1, the driver plays the role of the hiring manager and the executors play the role of the assistants. As you might have guessed, the HR department plays the role of cluster manager.

Driver: This is a program that is submitted to the cluster, to process Big Data using the Spark API. It can be written in Java, Scala, Python or R.

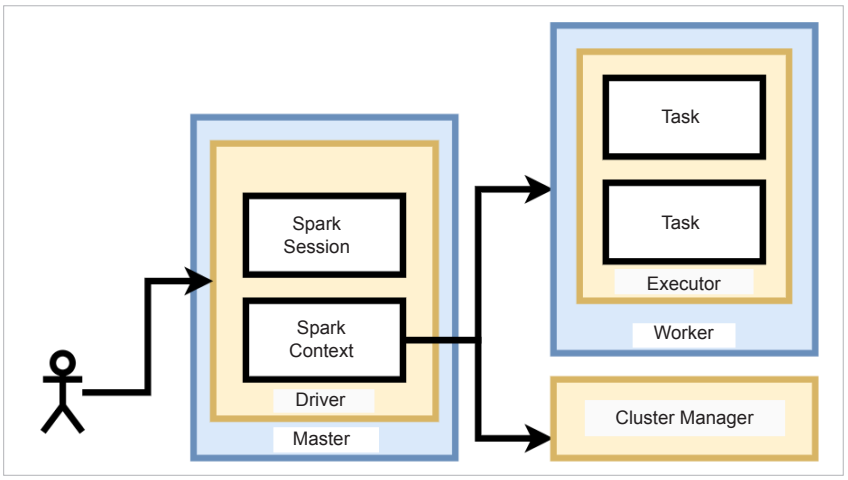

Figure 1: Apache Spark architecture

Spark context: The driver program that runs on the master node maintains the Spark context. Irrespective of the number of jobs that are submitted to the driver, it maintains only one context. Each job runs in a separate session, but all use the same context. The responsibilities of the context include analysing and dividing the job into smaller tasks with smaller data partitions, procuring workers from the cluster manager, allocating the tasks on the executors, and collating the responses from executors.

Executor: The worker nodes run one or more executors. Each executor may run multiple tasks as long as all of them belong to the same job. The executors of different jobs are inaccessible to each other. The responsibility of an executor is to run the task on the assigned data partitions and send the results back to the context.

Setting up Spark local

Though the real cluster is made up of several production machines, it can also be set up on a single development machine for experimentation. The only prerequisite is to have JDK, version 8 or later installed on the development machine. Since Apache Spark is written for JVM, the physical machine can be powered up by any operating system, though Linux is the most popular choice.

Once the development machine is ready with JDK, download the latest Apache Spark from *https://spark.apache.org/downloads.html*

Spark can also be downloaded from a Linux terminal with the following command:

```
wget https://www.apache.org/dyn/closer.lua/spark/spark-3.1.2/spark-3.1.2-bin-hadoop3.2.tgz
```

Copy the downloaded archive to a folder of your choice, if needed. Extract the archive with the following command:

```
tar -xvf spark-3.1.2-bin-hadoop3.2.tgz
```

It creates a folder by the name *spark-3.1.2-bin-hadoop3.2*. This folder will be referred to as *SPARK_HOME* hereafter.

For simplification, add the Spark binaries to the path with the following commands:

```
export PATH=$PATH:<SPARK_HOME>/bin
export PATH=$PATH:<SPARK_HOME>/sbin
```

Spark shell

Apache Spark comes with an interactive shell, which can be used during prototyping. The shell starts a Spark application with a Spark context on the specified master node. It accepts